Sequential Circuits and Latches

1. Introduction to Sequential Circuits

Combinational circuits only depend on the current input values to produce outputs. Sequential circuits, however, are digital circuits that store and use previous state information to determine their next state.

- Outputs in a sequential circuit are a function of both the current inputs and the present state of the storage elements.
- The general purpose of these circuits is to store information for an indefinite amount of time.

Applications:

Sequential circuits are commonly used in digital systems to implement:

- State machines
- Timers
- Counters
- Memory elements (implemented using flip-flops, which maintain their binary state even when inputs change)

2. Types of Sequential Circuits

Sequential circuits are divided into two primary categories based on how they handle time and state changes.

Synchronous vs. Asynchronous Circuits

Feature	Synchronous Sequential Circuit	Asynchronous Sequential Circuit
Clock Signal	Uses a clock to synchronize changes.	Does not use a clock; changes occur immediately with input changes.
Operation	All changes happen at specific time intervals (clock pulses).	Changes occur as soon as inputs change, without waiting for a clock.
Speed	Slower due to clock dependency.	Faster because it reacts immediately to input changes.

Feature	Synchronous Sequential Circuit	Asynchronous Sequential Circuit
Design Complexity	Easier to design and analyze because changes happen in sync.	More complex due to potential timing issues (hazards, glitches).
Reliability	More reliable and predictable.	Less reliable due to possible race conditions.

3. Introduction to Latches and Flip-Flops

Synchronous sequential circuits that use clock pulses as inputs for their storage elements are called **clocked sequential circuits**. The simplest form of storage element used in these circuits is the flip-flop.

- **Flip-Flop:** A binary storage device capable of storing one bit of information and having specific timing characteristics.
- **Latches:** The most basic storage elements from which flip-flops are usually constructed.
- Latches store a single bit of information and hold its value until it is updated by new input signals.
- They act as temporary storage elements and can be implemented using various digital logic gates, such as AND, OR, NOT, NAND, and NOR gates.

4. The S-R (Set-Reset) Latch

Set-Reset latches are the simplest form of latches and form the basic building blocks for all other types of flip-flops.

- It is implemented using two inputs: S (Set) and R (Reset).
- The S input sets the output to logic 1, while the R input resets the output to logic 0.
- They are also known as "preset" and "clear" states.
- A basic SR Latch consists of 2 cross-coupled NOR gates or 2 cross-coupled NAND gates, producing 2 outputs: Q and Q'.

Building SR Latch (Using NAND Gate):

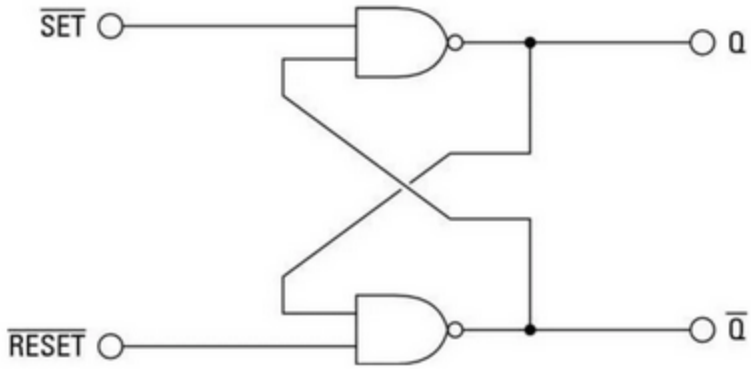
When built with NAND gates, the SR Latch becomes an Active-Low device (often denoted as $\overline{S}$-$\overline{R}$ Latch). This means that a logic 0 triggers the action, and logic 1 is the neutral resting state.

Logic Rules:

- **Hold State ($S' = 1, R' = 1$):** Both inputs are inactive (resting at High). The latch holds its previous value.

- **Set State ($S' = 0, R' = 1$):** Driving S' low forces Q to 1, which feeds back to make $Q' = 0$.
- **Reset State ($S' = 1, R' = 0$):** Driving R' low forces Q' to 1, which feeds back to make $Q = 0$.
- **Invalid State ($S' = 0, R' = 0$):** Both inputs dropping to 0 simultaneously forces both NAND gates to output a 1. Therefore, $Q = 1$ and $Q' = 1$, which violates the inverse rule and creates an undefined state.
  Truth Table:

S'	R'	Q (Next State)
0	0	Invalid (Undefined)
0	1	1 (Set)
1	0	0 (Reset)
1	1	Hold (No Change)



Building SR Latch (Using NOR Gate):
Truth Table When built with NOR gates, the SR Latch is an Active-High device. This means that a logic 1 triggers the Set or Reset action, while logic 0 is the neutral/resting state.

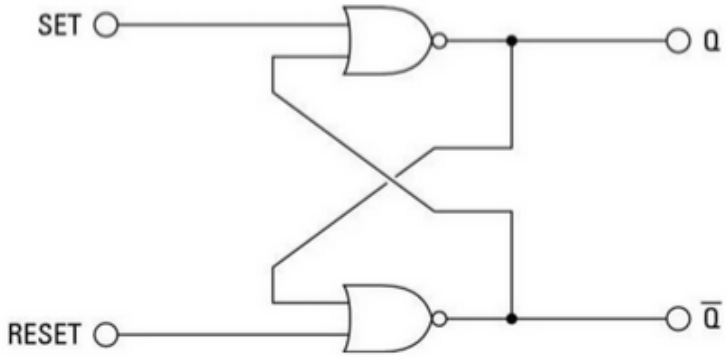
Logic Rules:

- **Hold State ($S = 0, R = 0$):** Both inputs are inactive. The cross-coupled feedback loop maintains whatever value was previously stored in Q.
- **Set State ($S = 1, R = 0$):** The S input forces Q' to 0, which feeds back to make $Q = 1$.
- **Reset State ($S = 0, R = 1$):** The R input forces Q to 0, which feeds back to make $Q' = 1$.
- **Invalid State ($S = 1, R = 1$):** Both inputs attempt to force a zero. This forces both Q and Q' to equal 0 simultaneously. This violates the rule that Q and Q' must be inverses, making it an undefined/invalid state.(Using NOR Gate):
  When built with NOR gates, the SR Latch is an Active-High** device. This means that a logic 1 triggers the Set or Reset action, while logic 0 is the neutral/resting state.

Logic Rules:

- **Hold State ($S = 0, R = 0$):** Both inputs are inactive. The cross-coupled feedback loop maintains whatever value was previously stored in Q.
- **Set State ($S = 1, R = 0$):** The S input forces Q' to 0, which feeds back to make $Q = 1$.
- **Reset State ($S = 0, R = 1$):** The R input forces Q to 0, which feeds back to make $Q' = 1$.
- **Invalid State ($S = 1, R = 1$):** Both inputs attempt to force a zero. This forces both Q and Q' to equal 0 simultaneously. This violates the rule that Q and Q' must be inverses, making it an undefined/invalid state.
  Truth Table:

S'	R'	Q (Next State)
0	0	Hold (No Change)
0	1	0 (Reset)
1	0	1 (Set)
1	1	Invalid (Undefined)



5. S-R Latch with Control Input

Latches can be modified by adding a control input (C), which acts as an enable signal or clock. These controlled latches are used to construct more advanced flip-flops.

Logic Rules:

- When $C = 0$: The control input disables the circuit. The state of the output does not change, regardless of the values of S and R.
- When $C = 1$: The latch operates normally.
- When $C = 1$ and $S = R = 0$: The state holds (no change).
- When $C = 1$ and $S = R = 1$: An undefined state occurs.

Truth Table (SR Latch with Control):

C	S	R	Next state of Q
0	X	X	No change
1	0	0	No change
1	0	1	$Q = 0$; Reset state
1	1	0	$Q = 1$; Set state
1	1	1	Undefined

6. The D (Data) Latch

The D Latch (also called a "Transparent Latch") was designed specifically to eliminate the undesirable undefined state present in the SR latch.

It simplifies the architecture down to only two inputs: D (Data) and C (Control). Internally, the D input is wired directly to the S terminal, and passed through an inverter to the R terminal. This guarantees that S and R can never be 1 at the same time.

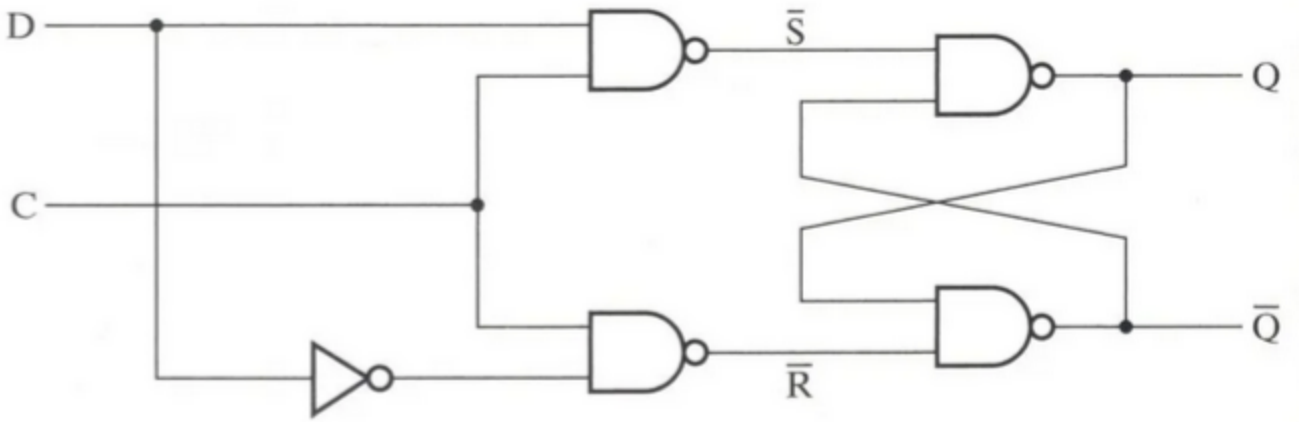
Logic Rules:

- The D input is only sampled when $C = 1$.
- If $D = 1$ (and $C = 1$), the Q output goes to 1 (Set state).
- If $D = 0$ (and $C = 1$), the Q output goes to 0 (Reset state).

Truth Table (D Latch):

C	D	Next state of Q
0	X	No change (Hold)
1	0	$Q = 0$; Reset state
1	1	$Q = 1$; Set state

The D Latch Circuit Diagram

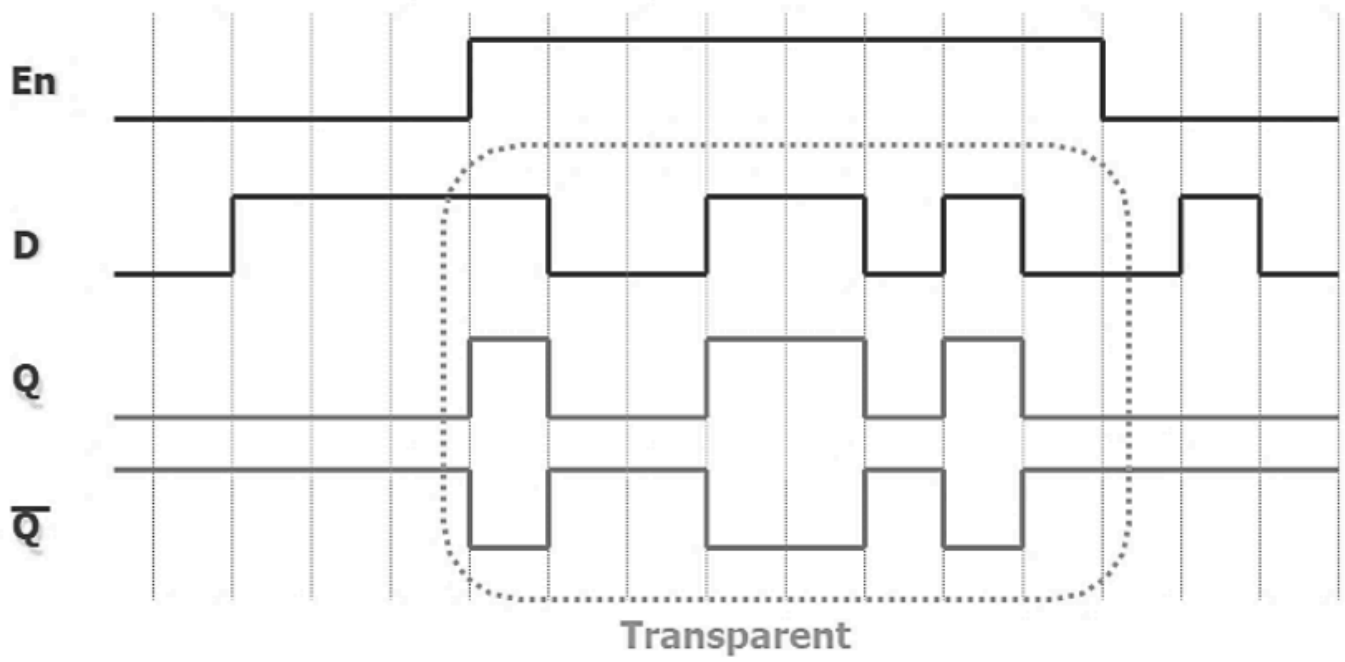


Transparent Latch

1. The D Latch as a "Transparent Latch"

As discussed previously, a D Latch uses a Control input (En or C) to sample data.

- When $En = 0$, the latch holds its value. It acts as a closed **Storage Cell**.
- When $En = 1$, the latch becomes **transparent**. This means the output Q instantly tracks any changes made to the input D. It acts like a straight wire.



The Clock Cycle

In synchronous digital systems, the Enable signal is driven by a continuous Clock Cycle.

- A clock cycle consists of one "up" phase (High/Enable) and one "down" phase (Low/Disable).
- The frequency is the number of these complete cycles per second.



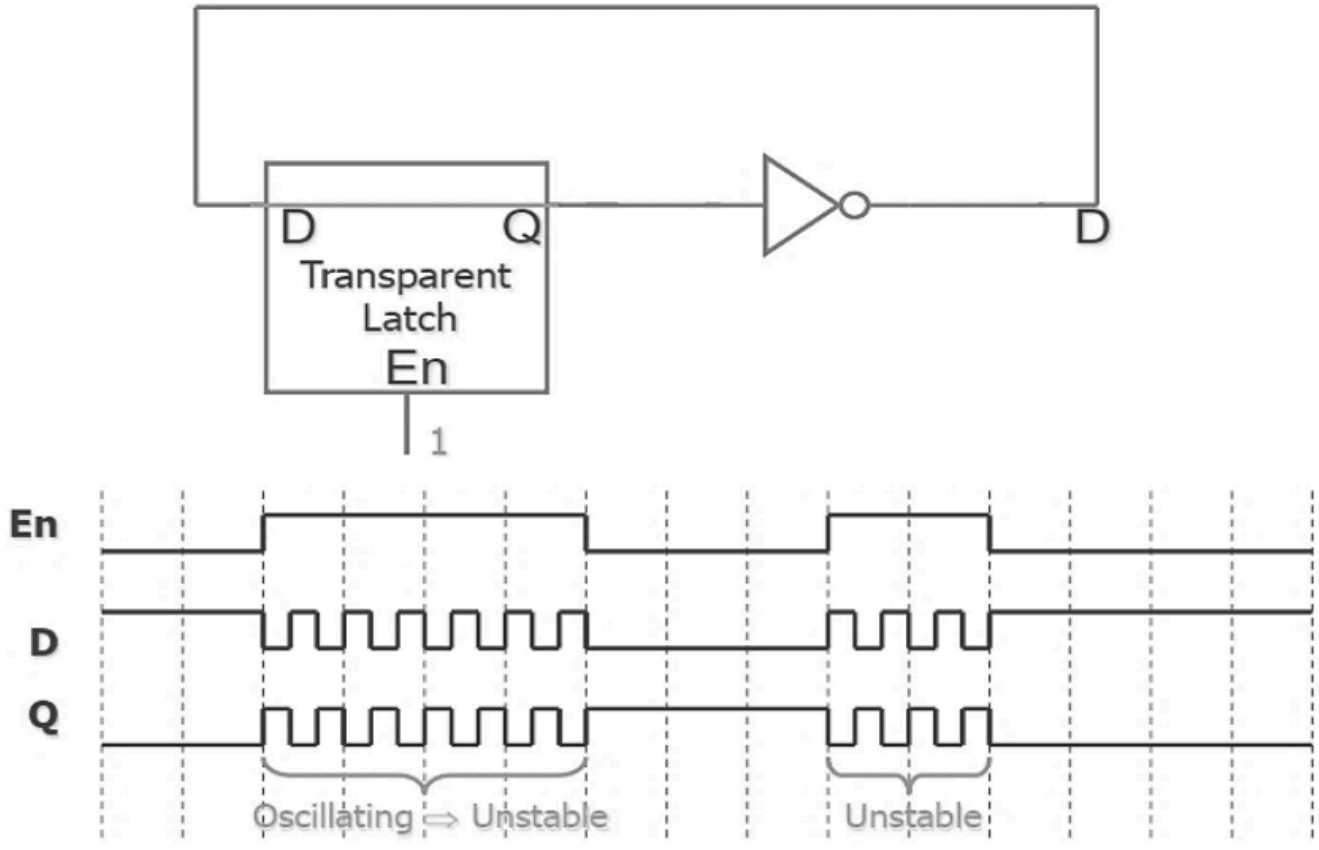
2. The Problem of Transparency

The defining feature of a D-Latch—its transparency during the High clock pulse—is also its biggest flaw in complex circuits.

The Setup: Imagine a scenario where the output Q of a Transparent D-Latch feeds into a block of combinational logic, and the result of that logic loops back to feed the input D of that same latch.

The Issue (Feedback Oscillation):

1. When the clock pulse is Low ($En = 0$), the latch works perfectly, holding its 1-bit value.
2. However, when the clock pulse goes High ($En = 1$), the latch becomes a transparent wire.
3. If the combinational logic is an **Inverter (NOT gate)**, the output Q is inverted and instantly fed back into D.
4. Because the latch is transparent, Q immediately changes to match the new D, which is then inverted again, fed back into D again, and so on.
5. This creates an **unstable oscillation loop** ($0 \to 1 \to 0 \to 1$) that continues endlessly as long as the clock remains High.
6. When the clock finally drops back to Low ($En = 0$), the latch will snap shut, but it may trap erroneous or random data depending on exactly when the clock fell during the oscillation.



3. The Solution: Flip-Flops

To solve the instability caused by transparency, engineers must completely isolate the input from the output during a clock cycle. This is achieved by creating a **Flip-Flop**.

What is a Flip-Flop?

- A Flip-Flop is essentially a combination of two separate latches wired together in series.
- The first latch is called the **Master** (often a D Latch), and the second is called the **Slave** (which can be an SR Latch or another D Latch).
- The clock signal driving the Slave is inverted from the clock signal driving the Master.

How it eliminates transparency: Because the clocks are inverted, only one latch can ever be open (transparent) at a given time.

- When the clock is High, the Master is open and accepts new input data, but the Slave is closed, preventing that data from reaching the final output.
- When the clock goes Low, the Master closes (locking in the data), and the Slave opens to safely pass that locked data to the final output.
- The input (D) is never directly connected to the final output (Q), completely eliminating the feedback oscillation problem.

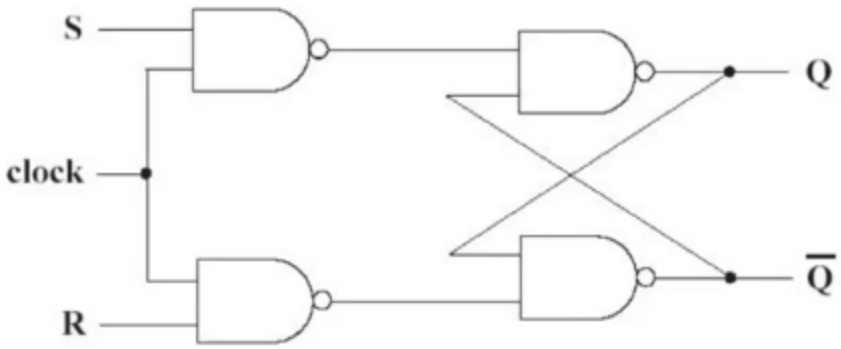
Sequential Circuits - The Clocked SR Flip-Flop

1. Clocked SR Flip-Flop (NAND Implementation)

To control when the SR Latch changes state, we add a clock (or enable) signal. By adding two "steering" NAND gates to the front of a basic active-low NAND latch, we create a Clocked SR Flip-Flop (often called a Gated SR Latch).

- The clock input (`Clk`) acts as a gatekeeper.
- When `Clk = 0` , the steering NAND gates output `1` regardless of S and R. Because the core latch is active-low, a `1` tells it to hold its state.
- When `Clk = 1` , the S and R inputs are inverted and passed through to the core latch, allowing it to function normally.

Circuit Diagram

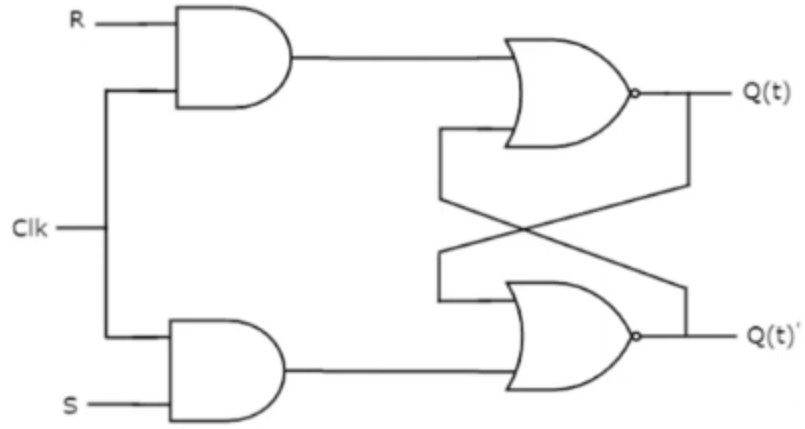


2. Clocked SR Flip-Flop (NOR Implementation)

To build a Clocked SR Flip-Flop around a NOR-gate core, we must use **AND gates** as the steering logic instead of NAND gates.

- The core NOR Latch is naturally an Active-High device (it requires a `1` to change state).
- When `Clk = 0` , the steering AND gates output `0` regardless of the S and R inputs. Supplying `0, 0` to a NOR latch tells it to Hold its state.
- When `Clk = 1` , the S and R inputs pass directly through the AND gates into the core latch without being inverted, triggering the Set or Reset actions normally.

Circuit Diagram



2. Fundamental Flip-Flop Tables

To fully analyze a sequential circuit, we use three specific types of tables. Each serves a different engineering purpose.

A. The Truth Table (Function Table)

The Truth Table defines the overarching behavior of the flip-flop based on the current inputs and the clock.

Clk	S	R	Q_{n+1} (Next State)
0	X	X	Q_n (Hold)
1	0	0	Q_n (Hold)
1	0	1	0 (Reset)
1	1	0	1 (Set)
1	1	1	Invalid

B. The Characteristic Table

The Characteristic Table is used for **analysis**. It explicitly details what the next state (Q_{n+1}) will be for every possible combination of the current state (Q_n) and the current inputs (S, R).

S	R	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	X (Don't Care/Invalid)
1	1	1	X (Don't Care/Invalid)

C. The Excitation Table

The Excitation Table is used for **design**. If you know what state you are currently in (Q_n) and you know what state you want to go to (Q_{n+1}), this table tells you what inputs (S, R) you must apply to make that transition happen.

Q_n	Q_{n+1}	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

(Note: X represents a "Don't Care" condition. For example, to transition from $0 \rightarrow 0$, S must be 0, but R can be either 0 or 1, so it is marked as X).

3. The Characteristic Equation

D. K-Map for the Characteristic Equation

To find the equation for the Next State (Q_{n+1}), we plot the outputs from the Characteristic Table onto a 3-variable K-Map based on the inputs S, R, and Q_n.

Note: The Invalid states where $S = 1$ and $R = 1$ are plotted as "Don't Cares" (X). This is a crucial engineering trick, as it allows us to use those cells to make larger groups and simplify the final hardware equation.

$S \backslash RQ_n$	00	01	11	10
0	0	1	0	0
1	1	1	X	X

Grouping the K-Map:

To get the most simplified equation, we circle the largest possible groups of `1`s (incorporating `X`s if they help make the group larger).

1. **Group 1 (The Horizontal Row):** Look at the entire bottom row ($S = 1$). It contains `1, 1, X, X`. We can group all 4 of these cells together. Because this group spans across all possible values of R and Q_n, those variables are eliminated.
   - **Resulting Term:** S
2. **Group 2 (The Vertical Column):** Look at the column where $R = 0$ and $Q_n = 1$ (the `01` column). It contains a `1` in the top row and a `1` in the bottom row. We group these 2 cells vertically. Because this group spans across $S = 0$ and $S = 1$, the S variable is eliminated. The constant variables for this column are $R = 0$ (which is R') and $Q_n = 1$ (which is Q_n).

- **Resulting Term:** $R'Q_n$

The Final Derived Equation:

By ORing (adding) the terms from our two groups, we get the minimized Characteristic Equation for the SR Flip-Flop:

$$Q_{n+1} = S + R'Q_n$$

Sequential Circuits - The JK Flip-Flop

1. SR vs. JK Behavior

The primary purpose of the JK Latch is to resolve the undefined/invalid state of the SR Latch. By crisscrossing the final outputs back to the input gates, the circuit uses its own current state to determine how to handle a `1, 1` input.

As shown , the function tables compare directly:

SR Latch Function:

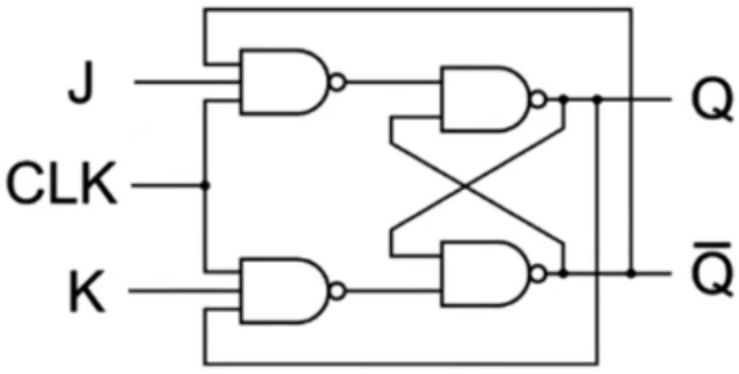
S	R	Q
0	0	Hold
0	1	0
1	0	1
1	1	Invalid

JK Latch Function:

J	K	Q_n
0	0	Hold
0	1	0
1	0	1
1	1	Toggle

2. Circuit Diagram (NAND Implementation)

This is the schematic you drew at the top of the page. Notice how it is essentially a clocked SR latch, but with the outputs Q and $\overline{Q}$ fed all the way back into the front steering NAND gates.



3. The Analytical Tables

These are the tables which explicitly map out the math behind the JK logic.

A. Characteristic Table

This table defines exactly what the next state (Q_{n+1}) will be based on the current inputs (J, K) and the current state (Q_n).

J	K	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

(Notice the last two rows: When $J = 1$ and $K = 1$, the next state is always the exact opposite of the current state Q_n. This is the Toggle feature in action).

B. Excitation Table

This table is used backward for circuit design. It tells you what inputs to apply to J and K if you want to force a specific state transition.

Q_n	Q_{n+1}	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

4. K-Map Derivation for the Characteristic Equation

To find the equation for the Next State (Q_{n+1}), we plot the outputs from the JK Characteristic Table onto a 3-variable K-Map based on the inputs J, K, and Q_n.

Unlike the SR Flip-Flop, the JK Flip-Flop has no "Invalid" states, meaning there are absolutely no "Don't Cares" (X) to use as wildcards. We must rely entirely on the definitive `1` s.

$J\backslash KQ_n$	00	01	11	10
0	0	1	0	0
1	1	1	0	1

Grouping the K-Map:

We have four `1`s on the board, and we can group them into two distinct pairs (loops of 2).

1. **Group 1 (The Bottom Corners):** Look at the bottom row ($J = 1$). There is a `1` in the `00` column and a `1` in the `10` column. Because K-Maps wrap around the edges, we can group these two corners together.
   - Across these two cells, K changes from 0 to 1, so it is eliminated.
   - The constant variables are $J = 1$ and $Q_n = 0$.
   - **Resulting Term:** $J\overline{Q}_n$
2. **Group 2 (The Vertical Column):** Look at the column where $K = 0$ and $Q_n = 1$ (the `01` column). There is a `1` in the top row and a `1` in the bottom row. We group these vertically.
   - Across these two cells, J changes from 0 to 1, so it is eliminated.
   - The constant variables are $K = 0$ and $Q_n = 1$.
   - **Resulting Term:** $\overline{K}Q_n$

The Final Derived Equation:

By ORing these two groups together, we get the minimized Characteristic Equation that governs the JK Flip-Flop:

$$Q_{n+1} = J\overline{Q}_n + \overline{K}Q_n$$

Sequential Circuits - Single-Input Flip-Flops (T & D)

While SR and JK flip-flops offer multiple control inputs, we often only need a circuit to do one of two specific things: act as a pure memory delay (D) or act as a frequency toggler (T). We can create these single-input flip-flops by mathematically modifying our existing dual-input circuits.

---

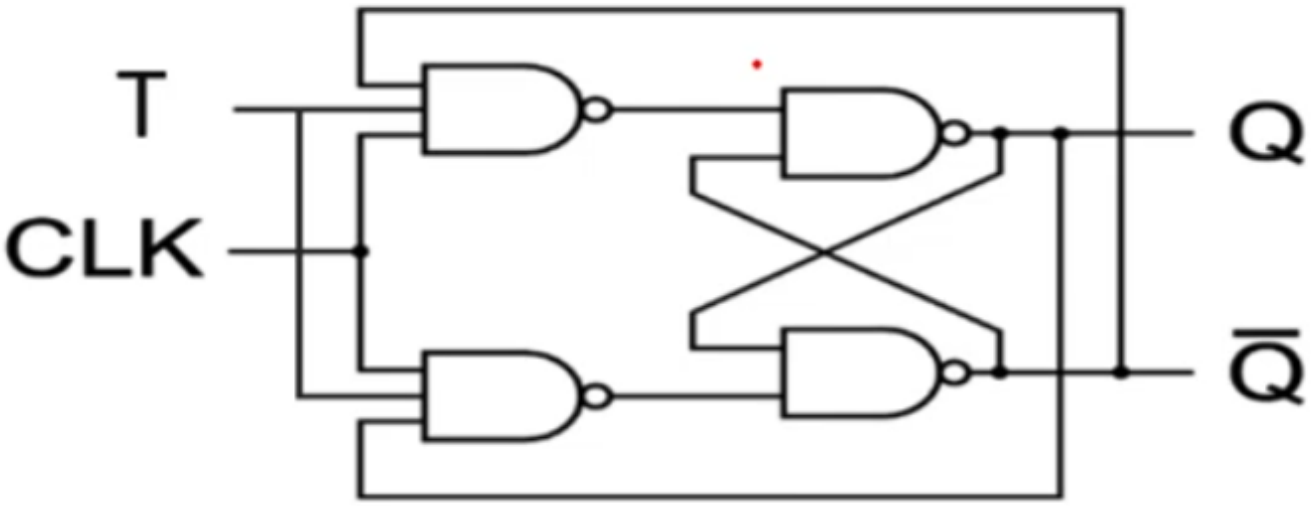
1. The T (Toggle) Flip-Flop

The T flip-flop is created by tying the J and K inputs of a JK Flip-Flop together into a single wire called T.

- If $T = 0$, both J and K receive `0`, meaning the circuit Holds its state.
- If $T = 1$, both J and K receive `1`, meaning the circuit Toggles its state on the clock edge.

Because holding $T = 1$ causes the output to invert on every single clock cycle, the T Flip-Flop is widely used to build binary counters and frequency dividers (it cuts the clock frequency exactly in half).

Circuit Diagram



Fundamental Tables

Truth Table (Function Table):

Clk	T	Q_{n+1}
0	0	Q_n (No Change)
1	1	$\overline{Q}_n$ (Toggle)

Characteristic Table:

T	Q_n	Q_{n+1}
0	0	0
0	1	1
1	0	1
1	1	0

Excitation Table:

Q_n	Q_{n+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

Characteristic Equation

If you look closely at the Characteristic Table, the output Q_{n+1} is `1` only when T and Q_n are *different*. This is the exact definition of an Exclusive-OR (XOR) gate.

$$Q_{n+1} = T \oplus Q_n$$

(Expanded: $Q*{n+1} = T\overline{Q}*n + \overline{T}Q_n$)

---

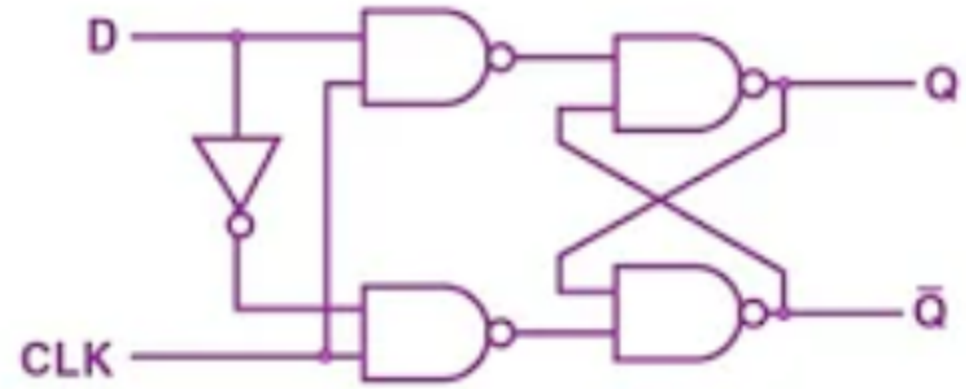
2. The D (Data/Delay) Flip-Flop

The D flip-flop is created by attaching an inverter (NOT gate) between the S and R inputs of an SR Flip-Flop, creating a single input D.

- This guarantees that S and R are always opposites, completely eliminating the Invalid $(1, 1)$ state.
- Whatever data is on the D pin is simply passed through to the Q output on the active clock edge.

Because it just captures and holds the input, it is the standard building block for computer memory (RAM) and data registers.

Circuit Diagram



Fundamental Tables

Truth Table (Function Table):

Clk	D	Q_{n+1}
0	X	Q_n
1	0	0
1	1	1

Characteristic Table:

D	Q_n	Q_{n+1}
0	0	0
0	1	0
1	0	1
1	1	1

Excitation Table:

Q_n	Q_{n+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

Characteristic Equation

Because the next state (Q_{n+1}) completely ignores the current state (Q_n) and simply copies the D input exactly, the equation is incredibly simple:

$$Q_{n+1} = D$$

Flip-Flop Conversions

The Golden Methodology (4 Steps)

To convert any flip-flop into another type, you are essentially designing a combinational logic circuit that sits in front of the **Available** flip-flop to make it behave like the **Desired** flip-flop.

You must always follow these 4 steps:

1. **Identify:** State what you *have* (Available FF) and what you *want* (Desired FF).
2. **The Conversion Table:** Create a master table. The inputs are the pins of the Desired FF and the current state (Q_n). Use the Desired FF's Characteristic logic to find the Next State (Q_{n+1}). Then, use the Available FF's Excitation Table to figure out what physical inputs are required to make that Q_{n+1} happen.
3. **K-Map Derivation:** Plot the required inputs onto K-Maps to find their simplified Boolean equations.
4. **The Circuit Diagram:** Draw the Available FF and wire its inputs using the logic gates derived in Step 3.

Conversion 1: SR to D Flip-Flop

- **Desired:** D Flip-Flop
- **Available:** SR Flip-Flop

Step 2: The Conversion Table

We use the D Flip-Flop logic ($Q_{n+1} = D$) to find the next state. Then we use the SR Excitation Table to figure out what S and R need to be to cause that transition.

Desired Input	Current State	Next State	Required Available Inputs	
D	Q_n	Q_{n+1}	S	R
0	0	0	0	X
0	1	0	0	1
1	0	1	1	0
1	1	1	X	0

Step 3: K-Map Derivation

From your conversion table, we plot the required inputs S and R onto separate 2-variable Karnaugh maps based on the inputs D and Q_n.

K-Map for S:

$D \backslash Q_n$	0	1
0	0	0
1	1	X

- **Grouping:** We group the `1` and the Don't Care (`X`) in the bottom row ($D = 1$).
- **Equation:** $S = D$

K-Map for R:

$D \backslash Q_n$	0	1
0	X	1
1	0	0

- **Grouping:** We group the `X` and the `1` in the top row ($D = 0$).
- **Equation:** $R = \overline{D}$

If we plot S and R onto separate 2-variable K-Maps (for D and Q_n), we get very simple loops:

- **For S:** A loop of 1s in the $D = 1$ row. Equation:

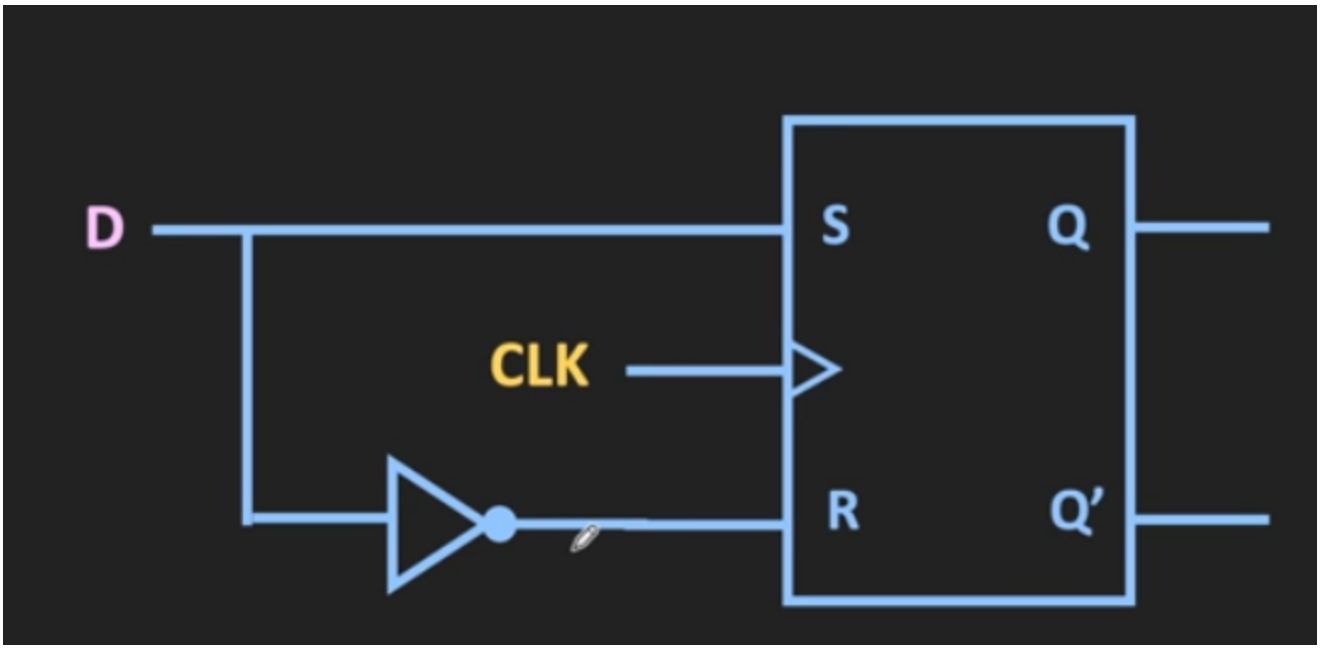
$$S = D$$

- **For R:** A loop of 1s in the $D = 0$ row. Equation:

$$R = \overline{D}$$

Step 4: Circuit Diagram

This mathematically proves the D-Latch/Flip-Flop architecture we covered previously. We simply attach an inverter to the D input and route it to R.



Conversion 2: T to JK Flip-Flop

- **Desired:** JK Flip-Flop
- **Available:** T Flip-Flop

Step 2: The Conversion Table

We use the JK Characteristic logic to determine Q_{n+1}. Then we look at the T Excitation logic (remember, $T = 1$ if the state changes, $T = 0$ if it holds) to find the required T input.

Desired Inputs		Current	Next State	Req. Input
J	K	Q_n	Q_{n+1}	T
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	1
1	0	1	1	0
1	1	0	1	1
1	1	1	0	1

Step 3: K-Map Derivation

We plot the required T input onto a 3-variable K-Map based on the desired inputs (J, K) and the current state (Q_n).

K-Map for T:

$J\backslash KQ_n$	00	01	11	10
0	0	0	1	0
1	1	0	1	1

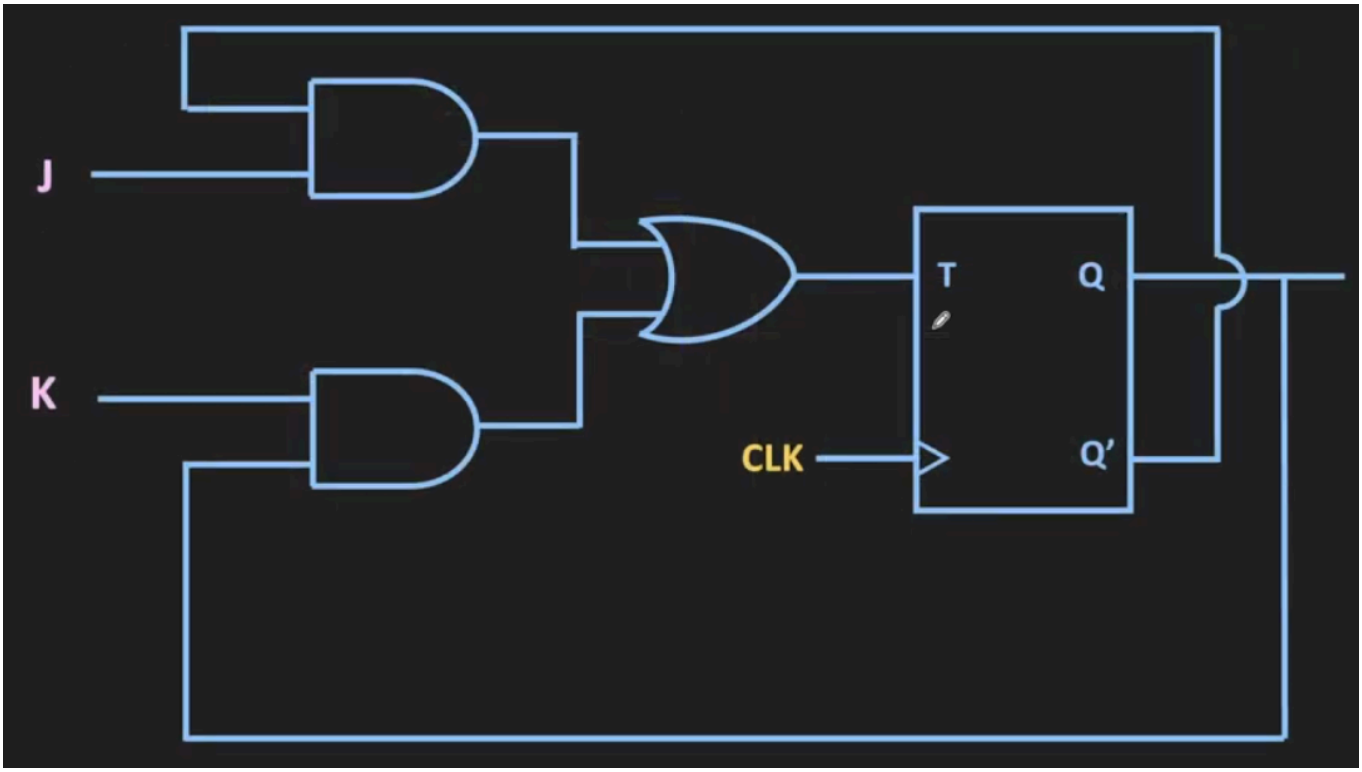
- **Grouping 1:** Group the `1`s at `100` and `110`. This eliminates K. The remaining term is $J\overline{Q}_n$.
- **Grouping 2:** Group the `1`s at `011` and `111`. This eliminates J. The remaining term is KQ_n.
- **Equation:** $T = J\overline{Q}_n + KQ_n$
- Group 1 groups the 1s where $J = 1$ and $Q_n = 0$.
- Group 2 groups the 1s where $K = 1$ and $Q_n = 1$.

Equation:

$$T = J\overline{Q}_n + KQ_n$$

Step 4: Circuit Diagram

We must use two AND gates and an OR gate to feed the correct logic into the T pin.



Conversion 3: SR to JK Flip-Flop

- **Desired:** JK Flip-Flop
- **Available:** SR Flip-Flop

Step 2: The Conversion Table

We map all 8 possible combinations of J, K, Q_n to find Q_{n+1}, and then use the SR Excitation logic to map out the required S and R inputs.

Desired Inputs		Current	Next State	Req. Inputs	
J	K	Q_n	Q_{n+1}	S	R
0	0	0	0	0	X
0	0	1	1	X	0
0	1	0	0	0	X
0	1	1	0	0	1
1	0	0	1	1	0
1	0	1	1	X	0
1	1	0	1	1	0
1	1	1	0	0	1

Step 3: K-Map Derivation

K-Map for S:

$J \backslash KQ_n$	00	01	11	10
0	0	X	0	0
1	1	X	0	1

- **Grouping:** We loop the `1`s at `100` and `110`. We *ignore* the `X`s in the `01` column because looping them wouldn't help us form a valid 4-cell group with the existing `1`s.
- **Equation:** $S = J\overline{Q}_n$

K-Map for R:

$J \backslash KQ_n$	00	01	11	10
0	X	0	1	X
1	0	0	1	0

- **Grouping:** We loop the `1`s at `011` and `111`. We ignore the `X`s in the `00` and `10` columns.
- **Equation:** $R = KQ_n$
- **For S:** We group the 1s and Xs where $J = 1$ and $Q_n = 0$Equation:

$$S = J\overline{Q}_n$$

- **For R:** We group the 1s and Xs where $K = 1$ and $Q_n = 1$Equation:

$$R = KQ_n$$

Step 4: Circuit Diagram

Look closely at this final diagram. By routing the outputs Q and Q' back to the front via AND gates, we have mathematically proven *exactly* how the original JK Flip-Flop was invented to solve the SR Flip-Flop's invalid state!

